12b — Domain Driven Design (DDD)

Key Concepts

- **DDD** = Design approach where code structure mirrors the real-world business problem
- Coined by Eric Evans (2003 book: *Domain-Driven Design*)
- Core idea: code should speak the language of the business, not the language of the database
- Two levels: **Strategic DDD** (how to divide the system) and **Tactical DDD** (how to model code inside each part)

Building Blocks

Strategic DDD

Concept	Definition
Ubiquitous Language	Shared vocabulary between devs and business — used in conversations AND in code
Bounded Context	Explicit boundary where a domain model is defined and consistent. Same word can mean different things in different contexts

Tactical DDD

Concept	Definition	Key Rule
Entity	Object with unique identity that persists over time	Two entities with same data but different IDs are still different
Value Object	Object defined entirely by its values, no identity	Immutable — replace, never modify. Two with same values are identical
Aggregate	Cluster of entities and value objects treated as one unit	All changes go through the Aggregate Root
Aggregate Root	Single entry point to an aggregate	External code can only reference the root, never internals directly
Domain Event	Something significant that happened — named in past tense	Decouples bounded contexts (OrderPlaced, PaymentReceived)
Repository	Abstraction that hides data access from the domain	Domain never sees SQL — asks for objects, repo fetches them

---

How It Works

Bounded Context — Same word, different models

```
┌──────────────────┐    ┌──────────────────┐    ┌──────────────────┐
│   Sales Context  │    │Warehouse Context │    │ Finance Context  │
│ Order =          │    │ Order =          │    │ Order =          │
│ customer intent  │    │ pick instruction │    │ billing record   │
└──────────────────┘    └──────────────────┘    └──────────────────┘
        ↕ communicate via Domain Events, not shared classes
```

Aggregate Structure

```
Order (Aggregate Root)
  ├── OrderLine (Entity)
  ├── OrderLine (Entity)
  └── ShippingAddress (Value Object)

External code → Order.AddItem()    ☑
External code → OrderLine.Update() ❌ (never directly)
```

Full Request Flow

```
[User Action]
    ↓
[Application Service] — orchestrates use case
    ↓
[Domain Model] — Entities/Aggregates enforce business rules
    ↓                    ↓
[Repository]       [Domain Events raised]
    ↓                    ↓
[Database]         [Other Bounded Contexts react]
```

Entity vs Value Object — Quick Distinction

	Entity	**Value Object**
Identity	Has unique ID	No ID
Equality	By ID	By values
Mutability	Can change over time	Immutable — replace, never modify
Examples	Order, Customer, Product	Money, Address, DateRange

Memory trick: Entity = *who* it is (ID matters). Value Object = *what* it is (values matter).

Industry Examples

Company	DDD Application
Uber	Separate bounded contexts: Pricing, Dispatch, Payments — each with its own model of a "Trip"
Amazon	Order aggregate enforces all business rules — no direct writes to order lines
Netflix	Content, Billing, Recommendations are separate bounded contexts — "Title" means different things in each
Banking	Account aggregate enforces overdraft rules — business logic lives in domain, not service classes

Interview Questions

Q1: What problem does DDD solve?

> Business logic scatters across services and models become anemic data bags. DDD puts logic back into the domain and makes code speak the language of the business.

Q2: What is Ubiquitous Language?

> Shared vocabulary between devs and business experts — used in conversations AND in code. Class names and methods use business terms, not technical jargon.

Q3: What is a Bounded Context?

> An explicit boundary where a domain model is consistent. Same word can mean different things in different contexts. In practice, each microservice maps to one bounded context.

Q4: Entity vs Value Object?

> Entity has unique ID — two with same data but different IDs are different. Value Object has no ID — defined by values, immutable, two with same values are identical.

Q5: What is an Aggregate Root?

> Single entry point to a cluster of entities and value objects. External code can only interact through the root — enforces business rules and consistency across the whole aggregate.

Q6: What are Domain Events and why are they useful?

> Past-tense facts that happened in the domain (OrderPlaced, PaymentReceived). Decouple bounded contexts — one context raises an event, others react without direct coupling.